

DartMinHash and the Dyadic Tiling Scheme

1 Goal

Let a sparse weighted set be represented as

$$x = \{(i, x_i) : x_i > 0\},$$

where i is a feature identifier and x_i is its nonzero weight. DartMinHash aims to construct a fixed-length sketch of x that preserves weighted Jaccard similarity. Instead of expanding a weighted item into many repeated unweighted tokens, DartMinHash samples from a continuous two-dimensional space associated with each nonzero feature.

For each feature i , define its dart sampling region as

$$R_i(\theta) = [0, x_i) \times \left[0, \frac{\theta}{\|x\|_1}\right),$$

where the horizontal axis is the weight coordinate and the vertical axis is the rank coordinate. A dart is a random point in this weight–rank plane. The smaller the rank, the more likely the dart is to survive in a MinHash bucket.

2 High-level DartMinHash procedure

DartMinHash consists of two stages.

1. **Dart generation.** A deterministic pseudo-random process generates darts in the union of the feature regions $R_i(\theta)$.
2. **Bucketed MinHash.** Each dart is mapped to one of k buckets using a hash function. In each bucket, only the dart with the smallest rank is retained.

The threshold θ controls how many darts are generated. If too few buckets are filled, θ is increased and more darts are considered.

3 The dyadic tiling idea

The central difficulty is that $R_i(\theta)$ is continuous and may have very different aspect ratios for different weights and ranks. A naive grid would either be too fine for large weights or too coarse for small weights.

DartHash solves this with a two-dimensional dyadic tiling scheme. The word *dyadic* means that the construction is organized by powers of two, not by binomial coefficients or a binomial distribution. The weight axis and rank axis are decomposed into exponentially growing layers.

For the weight direction, define layer index $\nu = 0, 1, 2, \dots$ with base point

$$w_\nu^{\text{base}} = \frac{2^\nu - 1}{t},$$

where t is the dart budget parameter. Thus the weight breakpoints are

$$0, \quad \frac{1}{t}, \quad \frac{3}{t}, \quad \frac{7}{t}, \quad \dots$$

For the rank direction, define layer index $\rho = 0, 1, 2, \dots$ with base point

$$r_\rho^{\text{base}} = 2^\rho - 1.$$

Thus the rank breakpoints are

$$0, \quad 1, \quad 3, \quad 7, \quad \dots$$

The layer pair (ν, ρ) defines a rectangular block in the weight–rank plane. This block is then subdivided into smaller cells. The width and height of each cell are chosen as

$$\Delta_w(\nu, \rho) = \frac{2^\nu}{t2^\rho}, \quad \Delta_r(\nu, \rho) = \frac{2^\rho}{2^\nu}.$$

Therefore every cell has the same area:

$$\Delta_w(\nu, \rho)\Delta_r(\nu, \rho) = \frac{2^\nu}{t2^\rho} \cdot \frac{2^\rho}{2^\nu} = \frac{1}{t}.$$

This invariant is the key to the method. Although cells have different shapes in different layers, their areas are identical. Hence each cell can use the same Poisson sampling rule.

4 Why the cell area invariant matters

In each small cell, the algorithm draws

$$Q \sim \text{Poisson}(1)$$

candidate darts. For each candidate dart, a pair of pseudo-random uniform numbers $(U_w, U_r) \in [0, 1)^2$ determines its position inside the cell:

$$w = w_0 + \Delta_w U_w, \quad r = r_0 + \Delta_r U_r.$$

The dart is accepted only if

$$w < x_i \quad \text{and} \quad r < \frac{\theta}{\|x\|_1}.$$

Because every cell has area $1/t$ and every cell receives a $\text{Poisson}(1)$ number of candidate darts, the construction behaves like a consistent Poisson dart process over the continuous weight–rank plane. The algorithm never materializes the full continuous process. It only enumerates the dyadic cells that can intersect the active region.

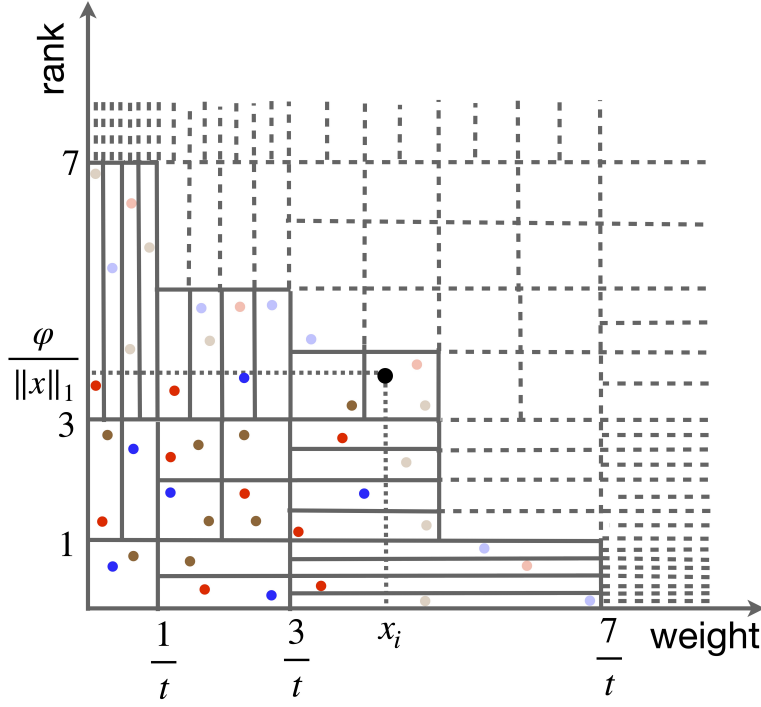


Figure 1: Dyadic tiling in the weight–rank plane. The horizontal axis is the weight coordinate and the vertical axis is the rank coordinate. Solid rectangles are the cells intersecting the active region $[0, x_i) \times [0, \varphi/\|x\|_1)$. Dashed rectangles indicate cells outside the current truncation boundary. The dyadic breakpoints $1/t, 3/t, 7/t, \dots$ and $1, 3, 7, \dots$ define exponentially growing layers.

5 Which layers are enumerated?

For feature i , the weight layers only need to cover $[0, x_i)$. Hence the largest weight layer is

$$\nu_{\max}(i) = \lfloor \log_2(1 + tx_i) \rfloor.$$

The rank layers only need to cover the current rank threshold

$$r_{\max} = \frac{\theta}{\|x\|_1}.$$

Thus

$$\rho_{\max} = \lfloor \log_2(1 + r_{\max}) \rfloor.$$

The algorithm enumerates

$$0 \leq \nu \leq \nu_{\max}(i), \quad 0 \leq \rho \leq \rho_{\max}.$$

This is why the method is efficient for sparse weighted sets: only nonzero features are visited, and for each feature the number of layers grows logarithmically with its weight and with the rank threshold.

6 Deterministic randomness

A crucial implementation detail is that the random process is not stored explicitly. Instead, every random choice is generated deterministically from hashes.

For a cell identified by

$$(i, \nu, \rho, a, b),$$

where a and b are the cell indices within the layer, the implementation forms a combined hash key. This key determines:

- the Poisson count Q in the cell;
- the within-cell coordinates (U_w, U_r) of each dart;
- the final dart identifier used for bucket assignment.

Thus the algorithm is random in distribution but deterministic as a function of the input and the hash seed. The same feature, layer, cell, and dart index always generate the same dart. This deterministic randomness is essential: two sets are compared inside the same hash-generated dart universe, so overlapping weighted regions produce identical darts with the correct probability.

7 Bucketed MinHash step

After generating accepted darts (d, r) , where d is the dart identifier and r is its rank, DartMinHash maps each dart to a bucket

$$j = h(d) \bmod k.$$

Each bucket keeps the dart with the smallest rank:

$$S_j(x) = \arg \min_{(d,r): h(d) \bmod k=j} r.$$

The final sketch is

$$S(x) = (S_1(x), S_2(x), \dots, S_k(x)).$$

If not all buckets are filled, the algorithm increases θ and repeats the dart-generation process with a larger rank threshold.

8 Algorithm

9 Why this is especially good for sparse weighted sets

The method is well matched to sparse weighted data for three reasons.

First, it only loops over nonzero coordinates. Empty dimensions do not participate at all.

Second, it avoids expanding weights into repeated tokens. A feature with large weight is handled by dyadic layers, so the cost grows with the logarithm of the weight scale rather than linearly with the weight.

Third, the same deterministic hash universe is shared across all sets. Therefore sketches can be compared directly without coordinating any additional randomness.

In short, dyadic tiling converts a continuous weighted sampling problem into a finite, logarithmic, hash-addressable enumeration of equal-area cells. This is the main reason DartMinHash can be both theoretically clean and practically efficient.

Algorithm 1 DartMinHash with dyadic tiling

Require: Sparse weighted set $x = \{(i, x_i) : x_i > 0\}$, sketch size k , dart budget t

```
1: Initialize  $S_j \leftarrow (\perp, +\infty)$  for  $j = 1, \dots, k$ 
2: Set  $\theta \leftarrow 1$ 
3: repeat
4:    $r_{\max} \leftarrow \theta / \|x\|_1$ 
5:   for all  $(i, x_i) \in x$  do
6:      $\nu_{\max} \leftarrow \lfloor \log_2(1 + tx_i) \rfloor$ 
7:      $\rho_{\max} \leftarrow \lfloor \log_2(1 + r_{\max}) \rfloor$ 
8:     for  $\nu = 0$  to  $\nu_{\max}$  do
9:       for  $\rho = 0$  to  $\rho_{\max}$  do
10:         $\Delta_w \leftarrow 2^\nu / (t2^\rho)$ 
11:         $\Delta_r \leftarrow 2^\rho / 2^\nu$ 
12:        Enumerate cells in the  $(\nu, \rho)$  block
13:        for all such cells  $C$  do
14:          Draw  $Q_C \sim \text{Poisson}(1)$  using the deterministic hash of  $C$ 
15:          for  $q = 1$  to  $Q_C$  do
16:            Generate  $(w, r)$  uniformly inside  $C$  using deterministic hashes
17:            if  $w < x_i$  and  $r < r_{\max}$  then
18:              Generate dart id  $d$ 
19:               $j \leftarrow h(d) \bmod k$ 
20:              if  $r < S_j.\text{rank}$  then
21:                 $S_j \leftarrow (d, r)$ 
22:              end if
23:            end if
24:          end for
25:        end for
26:      end for
27:    end for
28:  end for
29:  if some bucket is empty then
30:    Increase  $\theta$ 
31:  end if
32: until all buckets are filled
33: return  $S = (S_1, \dots, S_k)$ 
```
